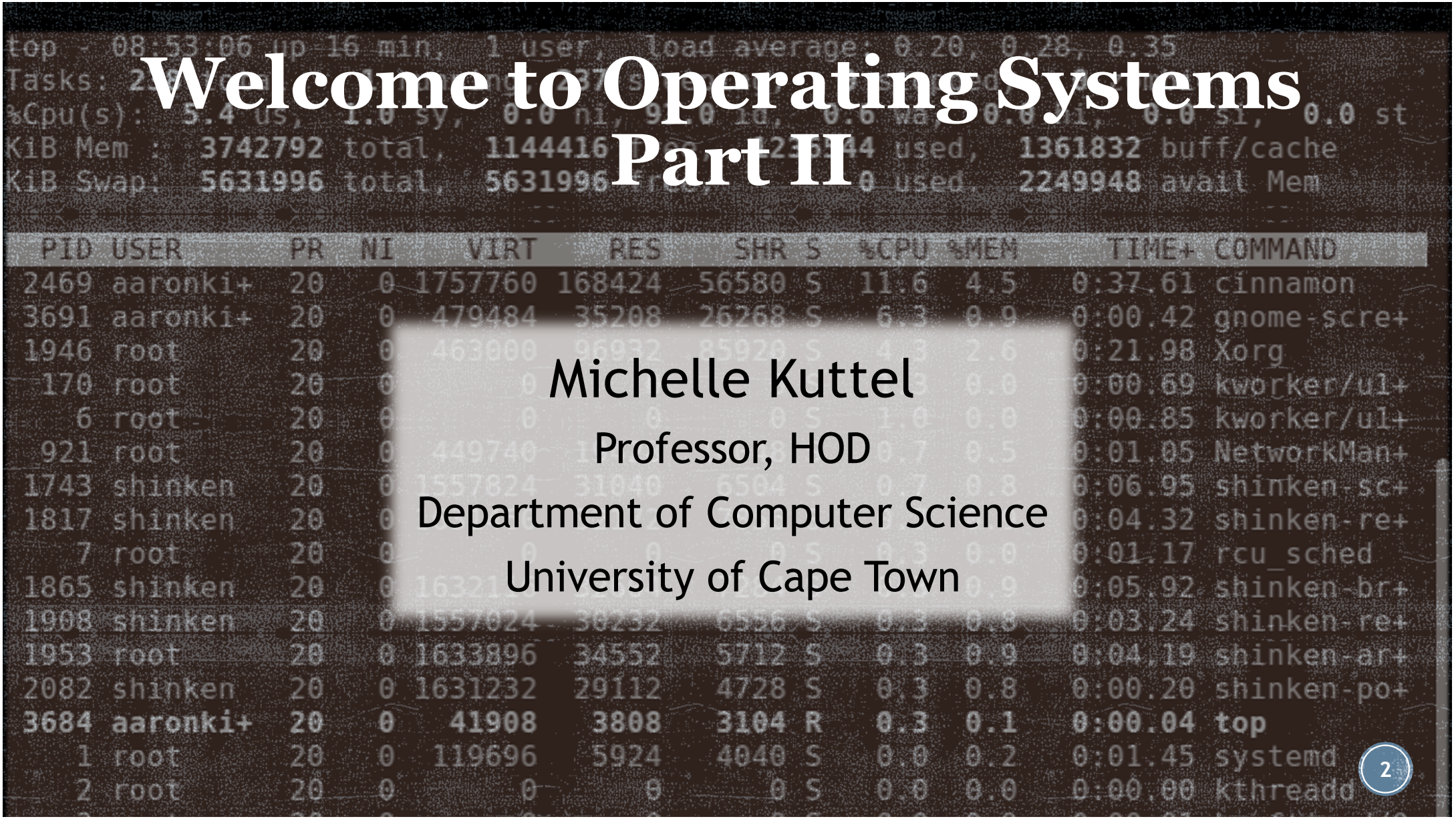


Welcome to Operating Systems Part II

Michelle Kuttel

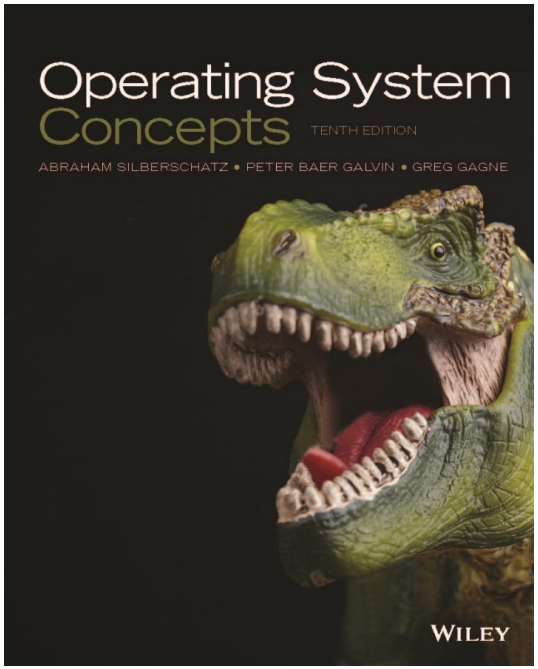
Professor, HOD

Department of Computer Science

University of Cape Town

Content to be covered

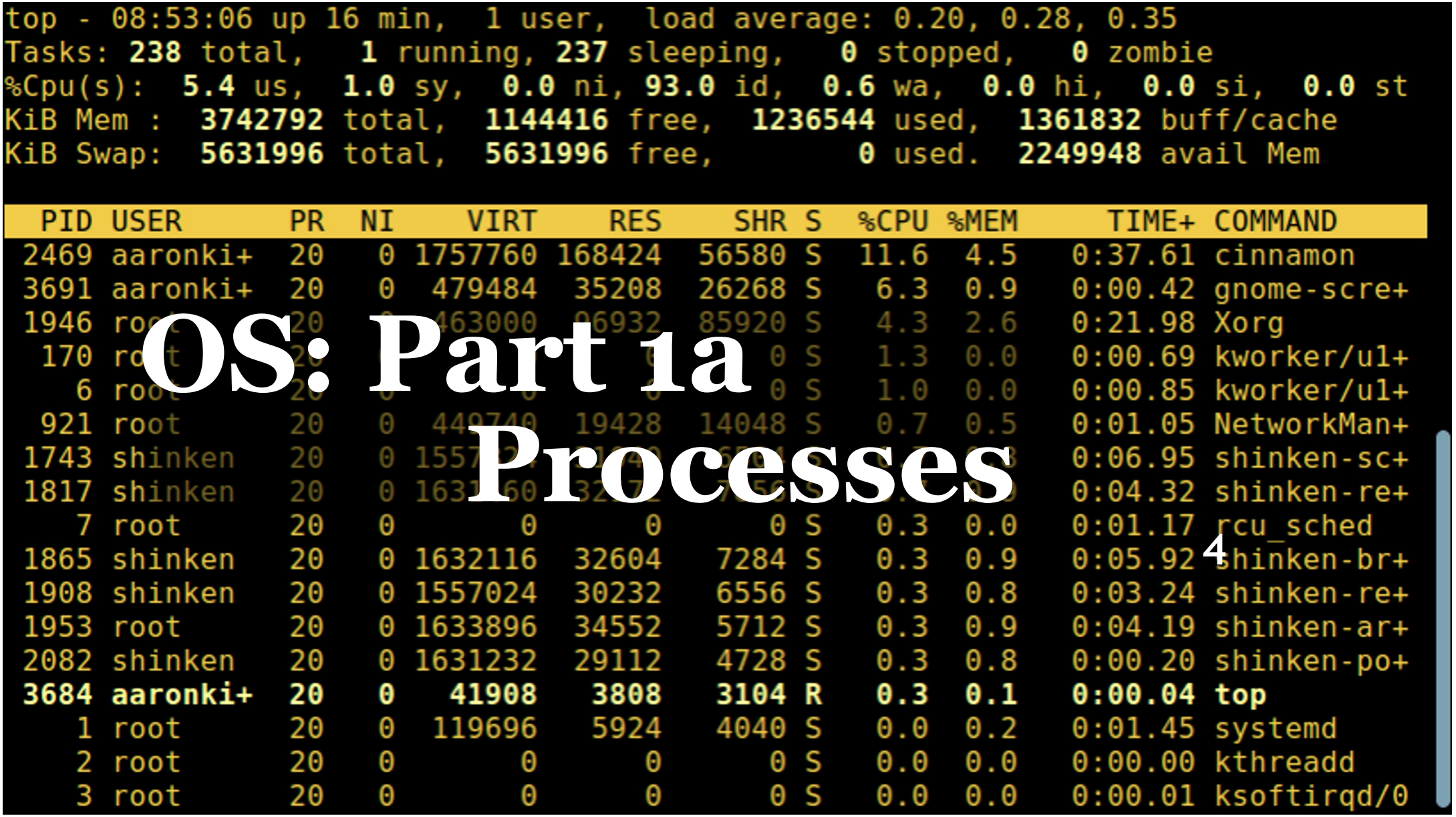
1. Processes
2. Process scheduling
   - The **assignment** is on process scheduling (and uses threads)
3. Threads
4. Synchronization
5. Deadlock



All content is **examinable** (unless otherwise specified).

I will give you book references when necessary, but I have used several books and other resources to prepare this content.

And it changes a bit every year…



OS: Part 1a Processes

Process: program in execution

A process is a fundamental concept for Operating Systems.

A **process** is an **executing program.**

- **A unit of work** in an OS – a ***job***



A **program** is **static;**

a **process** is ***dynamic***

Program vs Process vs Thread

Concept	Definition	Own memory/address space?	Own execution state?	Example
Program	A passive set of instructions stored on disk	No	No	`python`, `chrome`, `ls`
Process	A program in execution, with resources and protection	Yes	Yes	One running Chrome instance
Thread	A single path of execution within a process	No, shared with other threads in the same process	Yes	One worker thread inside Chrome

Program vs Process vs Thread

A **program** is a passive executable stored on disk.

static, not executing

A **process** is a running instance of a program.

Owns address space and resources.

Example
Program: python
Process: one running Python interpreter
Threads: main thread + worker threads

Thread: one execution path within a process. Executes instructions and is scheduled by the CPU.

Threads in the same process share

- code
- heap
- global data
- open files

Each thread has its own

- program counter
- registers
- stack

OS processes

This OS has one program that **runs at all times** on the computer – called **the kernel.**

Along with the kernel, there are two other types of programs that run:

- **System programs**: Used by the OS but are not necessarily part of the kernel.
- **Application programs**: All programs not associated with the operation of the OS

What is a kernel?

OS:

- Usually means the *complete system*
  - Including command interpreters, utility programs, window managers, …

Kernel:

- Core component of the system that manages resource access, memory, and **process scheduling** (this is the focus of this first section)

Process: program in execution

A modern Operating System (OS) has **many processes** executing **concurrently**
these are both OS (kernel) processes + user processes.

The OS manages processes:



OS also enables **inter-process communication.**

Process: program in execution

A user creates a process by double-clicking 

or running a program from the command line.

- **Two copies** of one **program** running in an OS are two **separate processes.**

> *but, most operating systems reuse memory wherever possible:* read-only code pages may be shared between processes when multiple copies of the program are executed at the same time

Process versus program

A process is a **program in execution** together with its **execution context** and **protected resources**.

A user process needs **permission** from the operating system kernel before e.g.:

- accessing the memory of any other process
- reading or writing to the disk
- before changing hardware settings

The **safe execution** of **untrusted code** is central to web browsers, virtual machines and operating systems.



Multiprocess architecture: web browsers

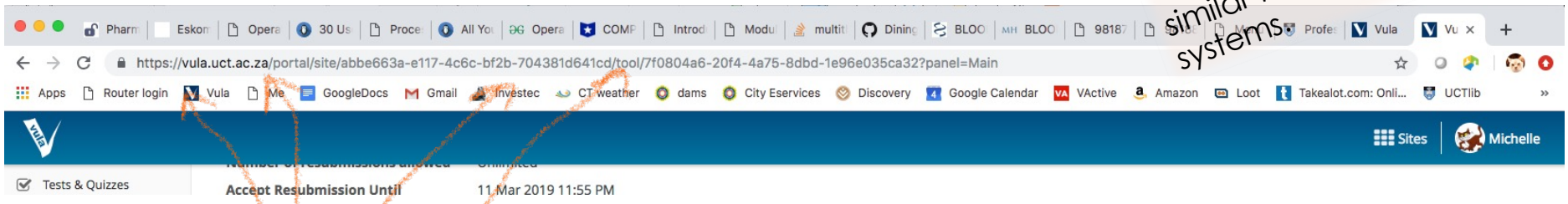
Modern web browsers such as Chrome, Internet Explorer, Firefox, and Safari, provide **process-like isolation** and **resource management** for web content, analogous to some OS responsibilities.

Browsers load and display web pages, but many pages **embed scripting programs** that the **browser must execute**.

- These scripts can be buggy or malicious; hackers have used them to take over vast numbers of home computers.

Multiprocess architecture: Chrome browser

Chrome uses process isolation similar to modern operating systems

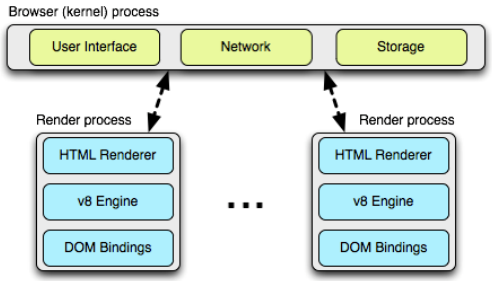


Tabbed browsing raises the issue that if any tab crashes, the entire browser (process) and all other tabs may crash too.

Google's Chrome browser addresses through a multi-process architecture comprising:

- one **browser process**
- many **sandboxed helper/content processes**

Advantage is that websites run in isolation from each other, a crash of a website only affects the renderer.



*sandbox provides increased security through restricted access to disk and network IO

`ps -A` On a Linux system, display every active process

PS(1) General Commands Manual PS(1)

NAME

ps - process status

SYNOPSIS

ps [**-AaCcEefhjlMmrSTvwXx**] [**-O** *fmt* | **-o** *fmt*]
[**-G** *gid*[,*gid*...]] [**-g** *grp*[,*grp*...]]
[**-u** *uid*[,*uid*...]] [**-p** *pid*[,*pid*...]]
[**-t** *tty*[,*tty*...]] [**-U** *user*[,*user*...]]
ps [**-L**]

DESCRIPTION

The **ps** utility displays a header line, followed by lines containing information about all of your processes that have controlling terminals.

A different set of processes can be selected for display by using any combination of the **-a**, **-G**, **-g**, **-p**, **-T**, **-t**, **-U**, and **-u** options. If more than one of these options are given, then **ps** will select all processes which are matched by at least one of the given options.

For the processes which have been selected for display, **ps** will usually display one line per process. The **-M** option may result in multiple output lines (one line per thread) for some processes. By default all of these output lines are sorted first by controlling terminal, then by process ID. The **-m**, **-r**, and **-v** options will change the sort order. If more than one sorting option was given, then the selected processes will be sorted by the last sorting option which was specified.

For the processes which have been selected for display, the information to display is selected based on a set of keywords (see the **-L**, **-O**, and **-o** options). The default output format includes, for each process, the process' ID, controlling terminal, CPU time (including both user and system time), state, and associated command.

The options are as follows:

-A Display information about other users' processes, including those without controlling terminals.

-a Display information about other users' processes as well as your own. This will skip any processes which do not have a controlling terminal, unless the **-x** option is also specified.

-C Change the way the CPU percentage is calculated by using a "raw" CPU calculation that ignores "resident" time (this normally has no effect).

-c Change the "command" column output to just contain the executable name, rather than the full command line.

-d Like **-A**, but excludes session leaders.

-E Display the environment as well. This does not reflect changes in the environment after process launch.

-e Identical to **-A**.

-f Display the uid, pid, parent pid, recent CPU usage, process start time, controlling tty, elapsed CPU usage, and the associated command. If the **-u** option is also used, display the user name rather then the numeric uid. When **-o** or **-O** is used to add to the display following **-f**, the command field is not truncated as severely as it is in other formats.

-G Display information about processes which are running with the specified real group IDs.

-g Display information about processes with the specified process group leaders.

-h Repeat the information header as often as necessary to guarantee one header per page of information.

-j Print information associated with the following keywords: **user**, **pid**, **ppid**, **pgid**, **sess**, **jobc**, **state**, **tt**, **time**, and **command**.

-L List the set of keywords available for the **-O** and **-o** options.

-l Display information associated with the following keywords: **uid**, **pid**, **ppid**, **flags**, **cpu**, **pri**, **nice**, **vsz=SZ**, **rss**, **wchan**, **state=S**, **paddr=ADDR**, **tty**, **time**, and **command=CMD**.

-M Print the threads corresponding to each task.

-m Sort by memory usage, instead of the combination of controlling terminal and process ID.

-O Add the information associated with the space or comma separated list of keywords specified, after the process ID, in the default information display. Keywords may be appended with an equals (`=') sign and a string. This causes the printed header to use the specified string instead of the standard header.

-o Display information associated with the space or comma separated list of keywords specified. Multiple keywords may also be given in the form of more than one **-o** option. Keywords may be appended with an equals (`=') sign and a string. This causes the printed header to use the specified string instead of the standard header. If all keywords have empty header texts, no header line is written.

-p Display information about processes which match the specified process IDs.

-r Sort by current CPU combination of cont process ID.

-S Change the way the by summing all exit parent process.

-T Display information to the device assoc input.

-t Display information to the specified te

-U Display the process specified real user

-u Display the process specified usernames

-v Display information following keywords: **pagein**, **vsz**, **rss**, **command**. The **-v** op option.

-w Use 132 columns to instead of the defa size. If the **-w** op than once, **ps** will necessary without r size. When output unlimited number of

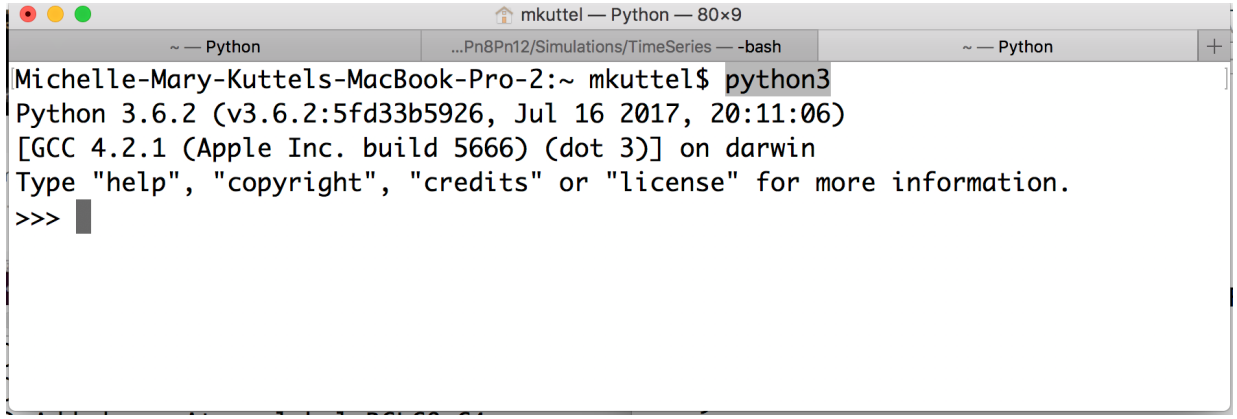
-X When displaying pro options, skip any p a controlling termi

-x When displaying pro options, include pr a controlling termi of the **-X** option. specified in the so use the one which w

Process: program in execution

Processes can **spawn other processes**,

and can also be **execution environment for other code.**

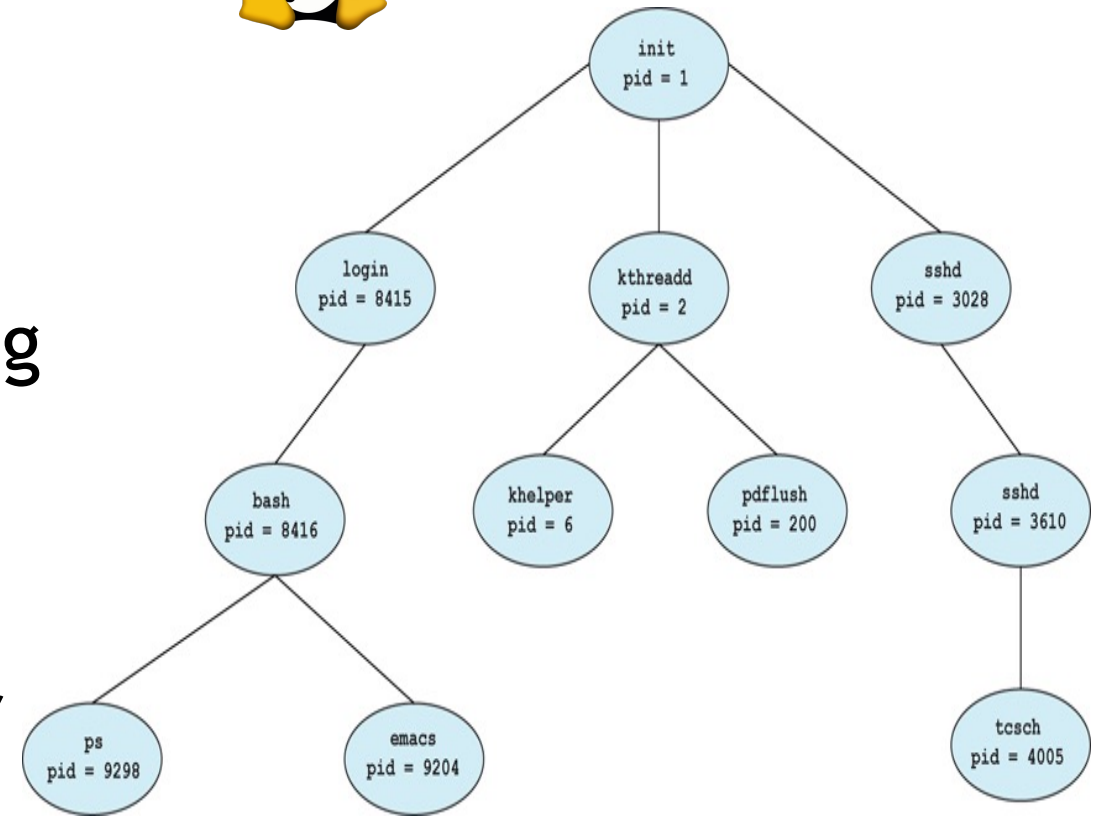


Process tree



Parent process creates **children** processes, which, in turn create other processes, forming **a tree of processes**

Identified and managed via a **process identifier** (PID)



Process tree

Parent process creates **children** processes, which, in turn, create other processes, forming **a tree of processes**

Identified and managed via a unique **process identifier** (PID)

On nightmare.cs.uct.ac.za:



```
ps --help all
```

```
ps -A --forest
```

```
ps -aef
```

```
pstree
```

Linux parent process

The `systemd` process is the **parent of all processes** on the system.
(`launchd` process in MacOS)

First program that is executed on boot.

Manages all other processes in the system.

Started by the kernel itself, so in principle it does not have a parent process.

Always has process ID of 1.

```
mkuttel@nightmare:~$ pstree
systemd─┬─agetty
        ├─151*[bash]
        ├─2*[bash───bash]
        ├─5*[bash───git───pager]
        ├─bash───joe
        ├─bash───driver.exe
        ├─2*[bash───make───java]
        ├─bash───5*[make───java]
        ├─bash───8*[make───java]
        ├─bash───cat
        ├─bash───2*[git───pager]
        ├─bash───3*[git───pager]
        ├─chronyd───chronyd
        ├─containerd───12*[{containerd}]
        ├─2*[cpptools───16*[{cpptools}]]
        ├─cron
        ├─dbus-daemon
        ├─dockerd───13*[{dockerd}]
        ├─inetd
        ├─irqbalance───{irqbalance}
        ├─4*[java───18*[{java}]]
        ├─lldpd───lldpd
        ├─10*[make]
        ├─master─┬─pickup
        │        ├─qmgr
        │        └─tlsmgr
        ├─mono───4*[{mono}]
        ├─multipathd───6*[{multipathd}]
        ├─mysqld───61*[{mysqld}]
        ├─networkd-dispat
        ├─nscd───35*[{nscd}]
        ├─packagekitd───2*[{packagekitd}]
        ├─polkitd───2*[{polkitd}]
        ├─postgres───6*[postgres]
        ├─puppet
        ├─python3.10───/opt/saltstack/───5*[{/opt/saltstack/}]
        ├─remote-dev-serv───launcher.sh───ld-linux-x86-64─┬─fsnotifier
        │                                                 └─66*[{ld-linux-x86-64
        ├─remote-dev-serv───launcher.sh───ld-linux-x86-64─┬─fsnotifier
        │                                                 └─70*[{ld-linux-x86-64
        ├─rpcbind
        ├─rsyslogd───3*[{rsyslogd}]
        ├─script───bash─┬─4*[grep]
        │               ├─6*[python3]
        │               └─python3───sh───java
        ├─script───bash
        ├─sh───node─┬─node───11*[{node}]
        │           ├─node───10*[{node}]
        │           └─10*[{node}]
        ├─sh───node─┬─node─┬─node─┬─java───21*[{java}]
        │           │      │      └─6*[{node}]
        │           │      └─11*[{node}]
        │           ├─node───12*[{node}]
        │           ├─node───10*[{node}]
```



Aside: bootstrapping process

When a computer boots up, the first program that runs is a tiny, simple **bootstrap loader.**

- The **bootstrap loader initializes** components of the system: e.g. CPU registers, device controllers, main memory etc.
- The **bootstrap loader stored** in NVM flash **memory firmware** on the motherboard and mapped to a specific memory location.
  - It can be updated by product manufacturers
  - Can be corrupted by viruses

The bootstrap loader loads a full bootstrap program into memory and executes it.

- Stored in the "**boot blocks**" at a fixed location in a secondary storage device. A device that has a boot partition is called a **boot disk** or **system disk**.
- The default Linux bootstrap loader is **grub2** (https://www.gnu.org/software/grub/manual/grub.html/).

The **full bootstrap program** loads **the entire operating system** (kernel) from a non-fixed location on the secondary storage device and starts the operating system running. The kernel:

- finds and mounts the root filesystem
- launches the first process on the system (init) which launches daemons on behalf of the system or users.

Definition: daemon

from the Greek "daimon”: “spirit" or "divine power."

A **daemon** is a process in an OS that *runs in the background* without user intervention.

e.g. on Linux:

sshd: SSH (Secure Shell) daemon responsible for secure remote shell access and encrypted communication between networked devices.

httpd (Apache): Web server daemon responsible for serving web pages, handling HTTP requests, and hosting websites.

rsyslogd: System logging daemon responsible for collecting, processing, and forwarding system log messages

cron: daemon responsible for scheduling and executing periodic tasks

When this command is run on a Linux server -

```
ps -fu msvmun001
```



- this output is produced:

```
UID          PID    PPID  C STIME TTY          TIME CMD
msvmun0+   16952       1  0 Mar02 ?        00:00:00 nano DESCRIPTION
msvmun0+  179773       1  0 Mar04 ?        00:00:00 SCREEN -
msvmun0+  179774  179773  0 Mar04 pts/132  00:00:00 /bin/bash
msvmun0+  185228       1  0 Mar04 ?        00:00:00 SCREEN -S Screen1
msvmun0+  185230  185228  0 Mar04 pts/157  00:00:00 /bin/bash
msvmun0+  220447       1  0 Mar01 ?        00:00:00 SCREEN -S
  virtual_screen1
msvmun0+  220448  220447  0 Mar01 pts/160  00:00:00 /bin/bash
```

Task: Draw **a process tree** labelled with the PID's that **correctly represents the relationships** between these processes

process ID maximum

the maximum number of processes that can exist simultaneously on a system



In the **Linux OS**, `/proc/sys/kernel/pid_max` contains the maximum pid
e.g. on CS nightmare server:

```
mkuttel@nightmare:~$ cat /proc/sys/kernel/pid_max
4194304
```

on uct's HPC head node:

```
mkuttel@srvcnthpc001 ~$ cat /proc/sys/kernel/pid_max
270336
```



In **MacOS**, the maximum PID is usually fixed at **99998**.



In **Windows**, there is no fixed maximum, restricted by the size of a word on the system.

Process ID wrapping

check_pid.sh

```
#!/bin/bash
pid=0
for i in {1..100000}; do
  : &
  if [ $! -lt $pid ]; then
    echo "Min pid: $!"
    echo "Max pid: $pid"
    break
  fi
  pid=$!
done
```

On my MacOS machine:

```
./check_pid.sh
Min pid: 100
Max pid: 99998
```

On my hpc head node (Linux):

```
./check_pid.sh
Min pid: 300
Max pid: 270335
```

: is the bash null command (always succeeds)& Creates background process, which gets a pid

The variable $! contains the process ID of the most recently executed background process. When true, wrapping has occurred

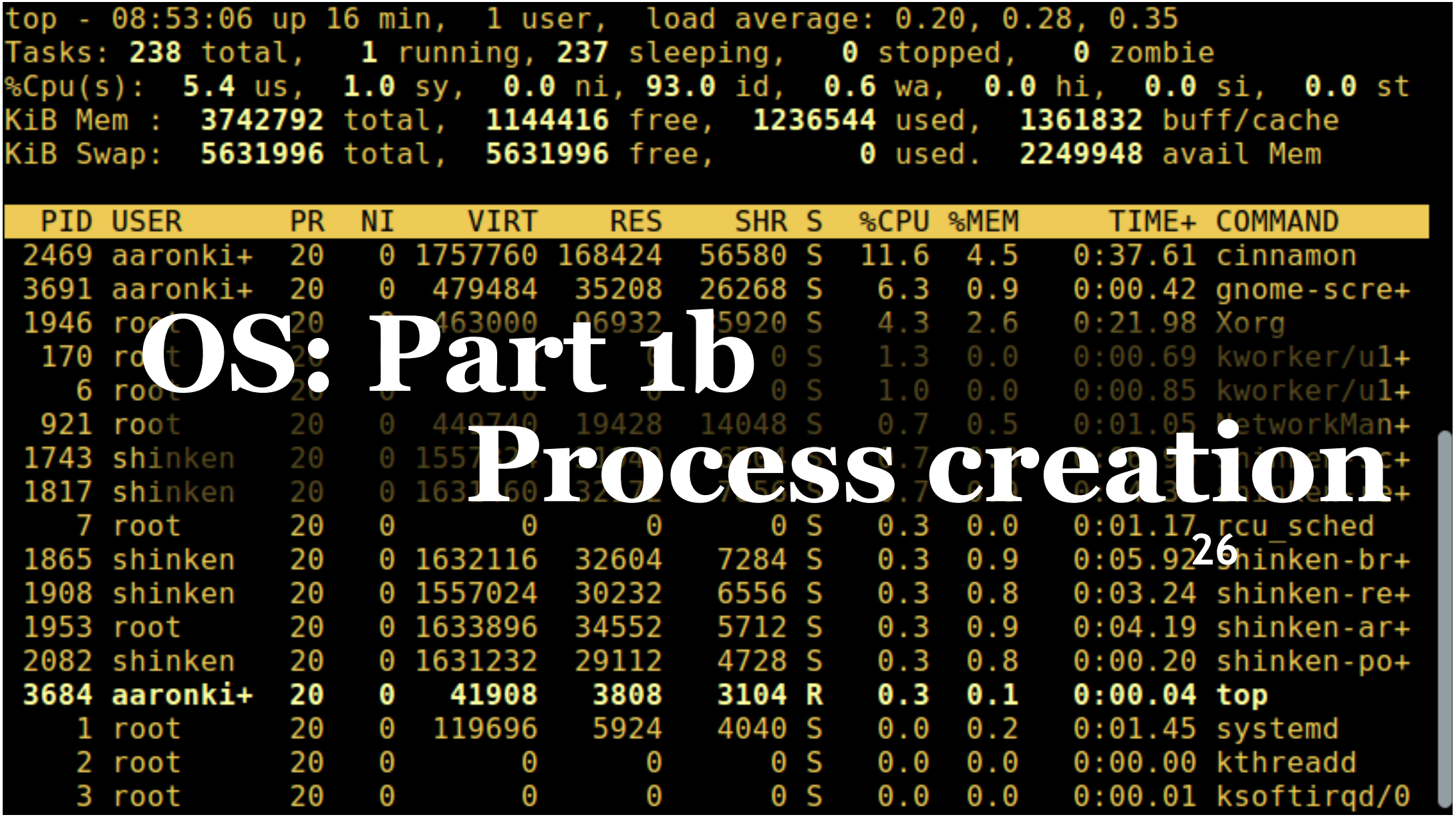
Process ID wrapping

Process IDs are usually **allocated sequentially**, beginning at a **minimum value** (often 100, or 300).

Once the **maximum value** is reached, allocation restarts, with any PIDs still assigned to processes skipped.

- This approach does have a slight **performance cost**, and **security issues**
- To improve security, another approach is to allocate PID's randomly within the range, also with a slight performance cost.

The allocation policy **is OS-dependent** and not something application code should rely on.



OS: Part 1b Process creation

Process Creation



Process context

A process is a program + **execution context** which encompasses:

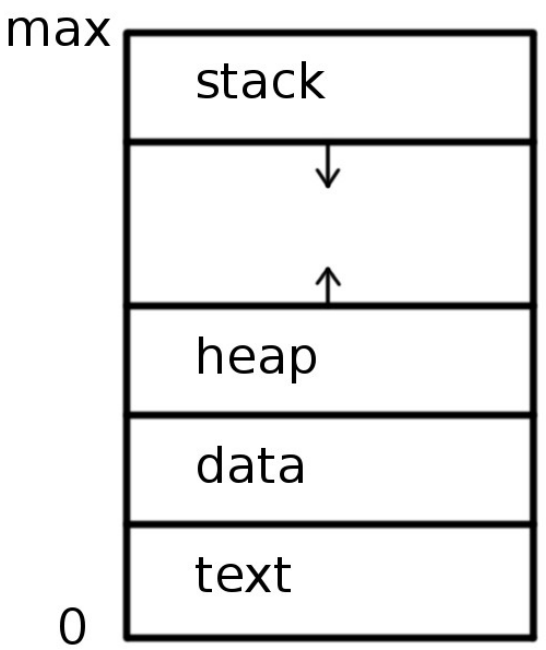
the **state of the processor** (processor context), which is the value of its program counter and all of its registers

process' **memory map**, which identifies the various regions of memory that have been allocated to the process.

A process in memory comprises:

- **Text** (compiled code)
- **Current state** (incl. program counter and registers)
- **Stack** (function parameters, return addresses, temporary data/local data)
- **Data** (initialized static and global variables)
- **Heap** (memory dynamically added during runtime – e.g. using *malloc* or *new*)

A process in memory.



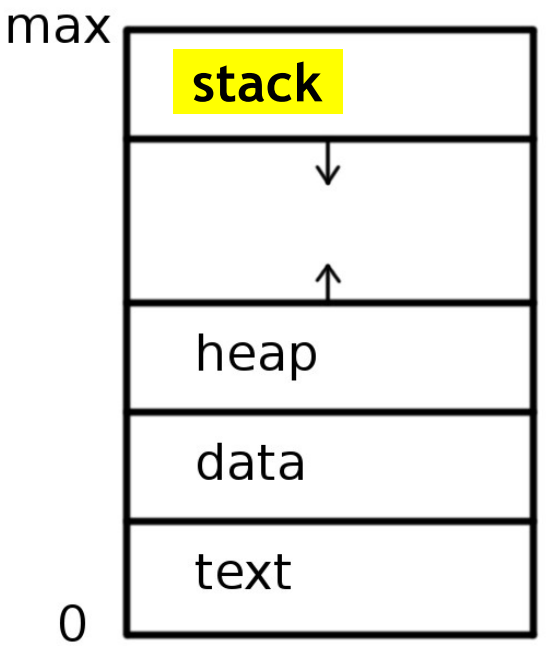
Process context

*The **stack** holds the state of local variables during procedure calls.*

- Used for **storing temporary data** (e.g. **local variables**) and **function call information**.
- Managed automatically by the system.
  - Local variables are automatically deallocated when they go out of scope (e.g., when a function returns).
- Limited size, faster access.
- Lifetime tied to variable scope.

Stack **size is fixed**, determined at the start of the program execution or by system settings. Exceeding the stack size can lead to a **stack overflow error**.

A process in memory.

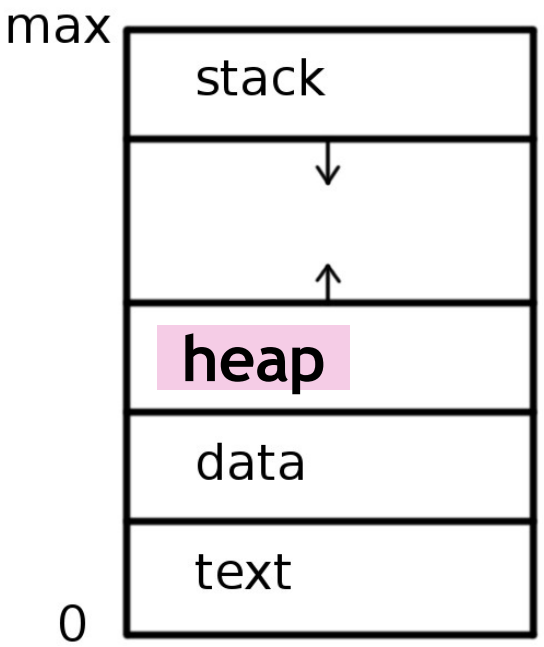


Process context

The **heap** *holds any dynamically allocated data structures the program might need.*

- Used for dynamic memory allocation.
- Managed manually by the programmer.
  - Memory allocation and deallocation on the heap are managed manually by the programmer using functions like malloc(), free(), new, and delete.
- Larger size, slower access.
- Lifetime independent of variable scope.

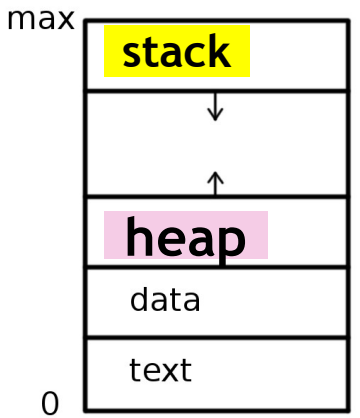
A process in memory.



Question: What happens when heap and/or stack become full?

Stack: function parameters, return addresses, temporary data/local data

Heap: memory dynamically added during runtime – e.g. using *malloc* or *new*



`Java: StackOverflowError`

Stack overflow occurs if the call stack pointer exceeds the stack bound -> usually program crash

Happens e.g. with infinite (or very deep) **recursion**

size of stack depends on programming language, machine architecture, multi-threading, amount of available memory etc.

- *More common with multithreading, because threads have reduced stack*

`Java: OutOfMemoryError`

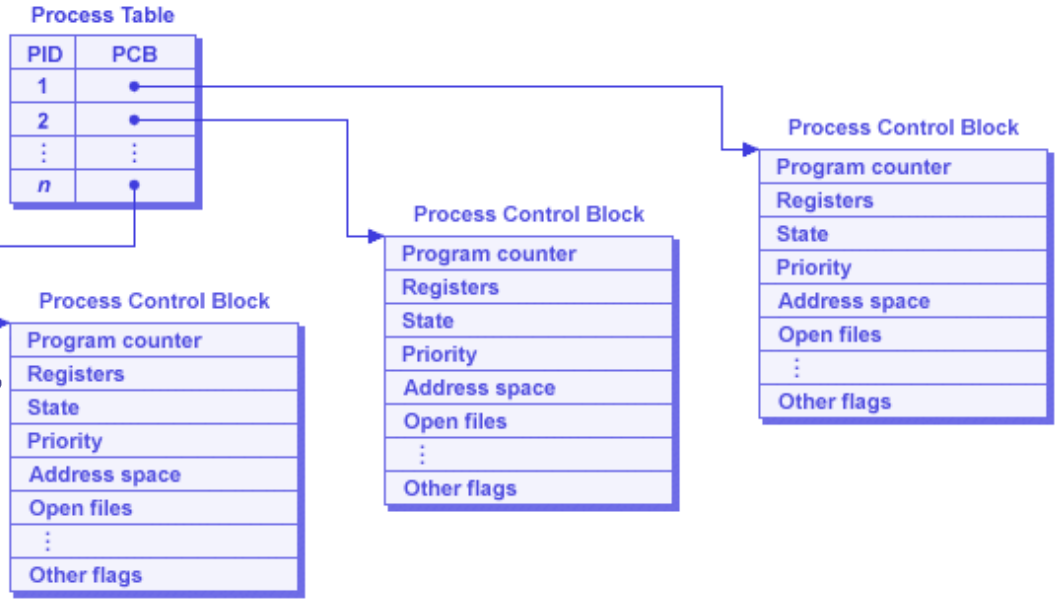
Heap exhaustion occurs when a process runs out of heap memory, usually because it allocates too many or too-large objects.

Virtual memory may delay failure, but allocation can still fail once limits are reached.

Creating a very large number of threads can contribute to memory exhaustion, although per-thread stack space is often the bigger issue.

Garbage collection in Java runs on heap memory.

Process list/table



To keep track of processes, the OS **maintains** a **process table** (or list).

Each entry is a

Process Control Block (PCB)

- Represents a particular process
- Has fields with information for the kernel about the process.

A process is identified by a unique **process ID (PID).**

Some operating systems (UNIX-derived systems) also have a **process group.**

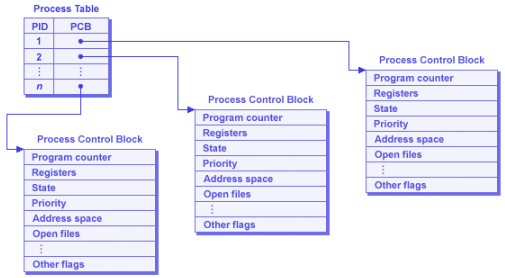
- A way to lump processes together so that related processes can be signalled.
- Every process is then a member of some process group.

Process Control Block (PCB)

Information associated with each process

Process state –
- running, waiting, etc

Program counter –
- location of next instruction to execute



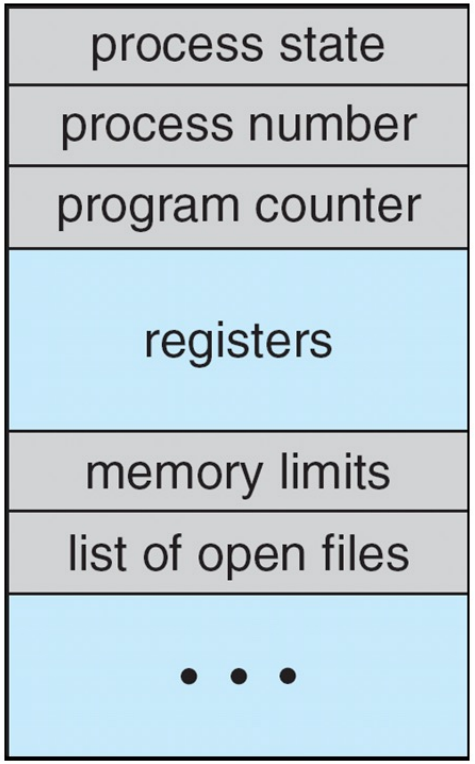
CPU registers –
- contents of all process centric registers, number and type is architecture dependent

CPU scheduling information –
- priorities, scheduling queue pointers

Memory-management information –
- memory allocated to the process

Accounting information –
- CPU used, clock time elapsed since start, time limits

I/O status information –
- I/O devices allocated to process, list of open files



In a multithreaded OS, per-process state and per-thread state are separated; each thread has its own execution context, while threads in a process share the address space and many resources.

Process representation in Linux/Darwin

The process block is the most **complicated data structure** in the operating system.

In **Linux**, the PCB is implemented as the data structure `task_struct`,

In **Darwin** (the core OS MacOS = kernel(XNU) + libraries +)

the process block is implemented as a data structure `task`

Both contain similar information about a process, such as process id, process state, file information, and so on.

For example, `task_struct` contains 83 field variables; 60 are primitive types, 3 are composite data structures, and 20 are pointers to composite data structures.

https://github.com/apple/darwin-xnu/blob/master/osfmk/kern/**task.c**

Process creation in Unix: fork()

UNIX has a **system call** to allow a **process to spawn another process**

`fork()` system call **creates a new process**

`fork()` = "clone me"

hello_fork.c

```
int main(void)
{
    pid_t pid = fork();

    printf("Hello from process, child PID = %d\n",pid);

    if (pid == -1) {
        perror("fork failed");
        exit(0);
    }
    else if (pid == 0) {
        printf("Bye from the child process! \n");
        _exit(1);
    }
    else {

        int status;
        (void)waitpid(pid, &status, 0);
    }
    printf("Bye from the parent process!\n");
    return 1;
}
```

What happens after fork()?

Processes have:

- same code after the fork,
- different return values from fork(),
- same open files unless changed,
- different PID,
- Same (usually copy-on-write) address space,

Parent and child run **concurrently** in **unspecified order**.

```c
int main(void)
{
    pid_t pid = fork();

    printf("Hello from process, child PID =
    %d\n",pid);

    if (pid == -1) {
        perror("fork failed");
        exit(0);
    }
    else if (pid == 0) {
        printf("Bye from the child process! \n");
        _exit(1);
    }
    else {

        int status;
        (void)waitpid(pid, &status, 0);
    }
    printf("Bye from the parent process!\n");
    return 1;
}
```

Details on UNIX process creation

fork() system call

- creates and initializes a new PCB
  - initializes kernel resources of new process with resources of parent (e.g., open files)
  - initializes program counter to be same as parent
- creates a new address space
  - initializes new address space with a copy of the entire contents of the address space of the parent
- places new PCB on the ready queue

the fork() system call "**returns twice**"

- once into the parent, and once into the child
  - returns **the child's PID** to the **parent**
  - returns 0 to the child

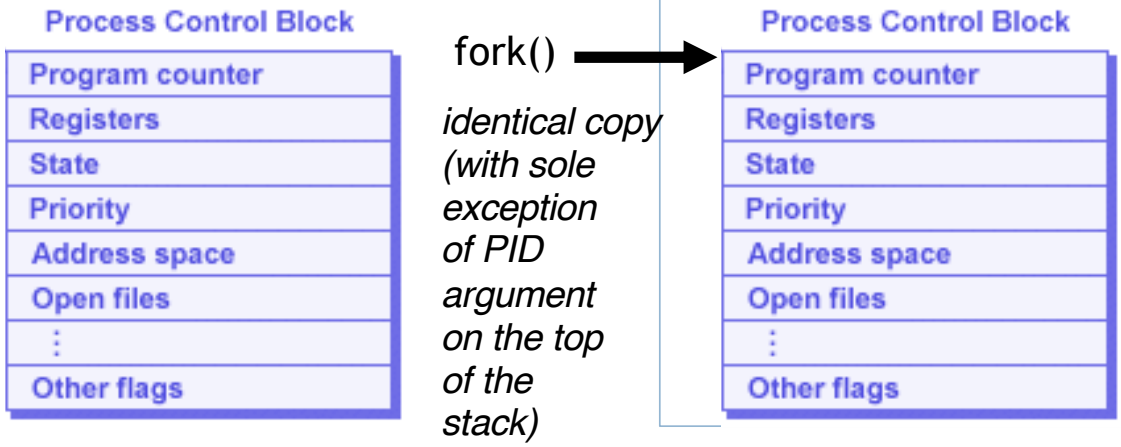
How many greetings?

gcc how_many_fork.c -o how_many_fork

./how_many_fork

```c
#include <stdio.h>


int main(void)
{
    printf("This is the start of the greetings\n");
    fork();
    printf("Hello!\n");
}
```



And now how many greetings?

gcc how_many_fork2.c -o how_many_fork2

./how_many_fork2

```c
#include <stdio.h>


int main(void)
{
    printf("This is the start of the greetings\n");
    fork();
    printf("Hello!\n");

     fork();
    printf("Hello!\n");

}
```

And now how many greetings?

gcc how_many_fork_more.c
  -o how_many_fork_more

./how_many_fork_more

Now, *draw a process tree*

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

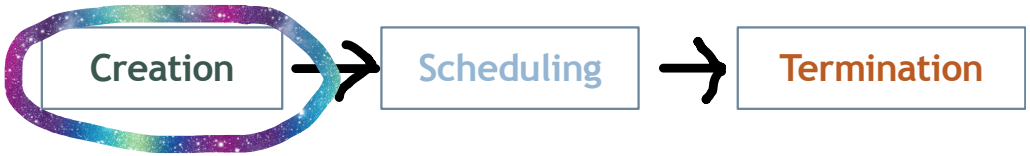
int main(void)
{
     long x=(long)getpid(); // get this process's PID
     printf("This is the start of the greetings\n");
     printf("Hello, %ld!\n",x);

     fork();
     x=(long)getpid();
     printf("Hello, %ld!\n",x);

     fork();
     x=(long)getpid();
     printf("Hello! %ld!\n",x);

     sleep(5);
     printf("Goodbye, %ld!\n",x);
     return 0;
}
```

Process creation



Resource sharing options for processes

- Parent and children **share all resources**
- Children **share subset** of parent's **resources** (e.g. **Linux**)
  - child is created with a new address space initially copied (copy-on-write) from the parent; exec() often called to run a new program
- Parent and child **share (almost) no resources** (e.g. **Windows**)
  - child is created with a new address space running a specified program image.

Linux separates **process creation** (fork) from **program loading** (exec), while Windows typically does both together in one step (CreateProcess).

Execution options

- Parent and children execute concurrently
- Parent waits until children terminate

Address space

- Child duplicate of parent - behaviour of `fork` system call in UNIX
- Child has a program loaded into it - behaviour of `spawn` system call in Windows

Process creation in Unix: how do we start a new program, instead of just forking the old program?

`exec()` system call used after a `fork` to **replace the process' memory space with a new program**

First fork, then exec

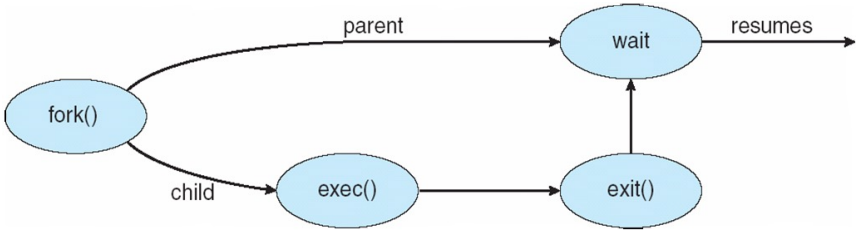
`int exec(char * prog, char * argv[])`

– loads program 'prog' into the address space

• i.e., over-writes the existing process image

– initializes hardware context, args for new program

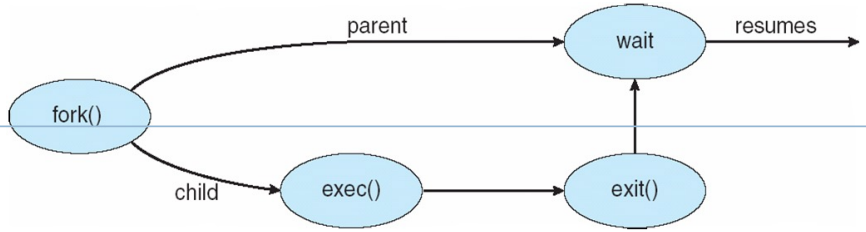
note: does not create a new process!



Exec basically wipes out the calling process, replacing it with the new one.

- the process continues **as the same process,** with the same PID, but running a **different program** image.

Process creation in Unix: exec()



`exec()` system call used after a `fork` to **replace the process' memory space with a new program**

Exec basically wipes out the calling process, replacing it with the new one.

hello_exec.c

```c
int main(void)
{
    pid_t pid = fork();


    printf("Hello from main process, child  PID = %d\n",pid);

    if (pid == -1) {
        perror("fork failed");
        exit(0);
    }
    else if (pid == 0) {
        printf("Forked child %d, about to be replaced with ls\n",pid);
        execlp("/bin/ls", "ls", NULL);
        exit(1);
    }
    else {

        int status;
        (void)waitpid(pid, &status, 0);
    }
    printf("Bye from the parent process! child done PID %d\n",pid);
    return 1;
}
```

exec()

hello_exec2.c

```c
int main(void)
{
    pid_t pid = fork();

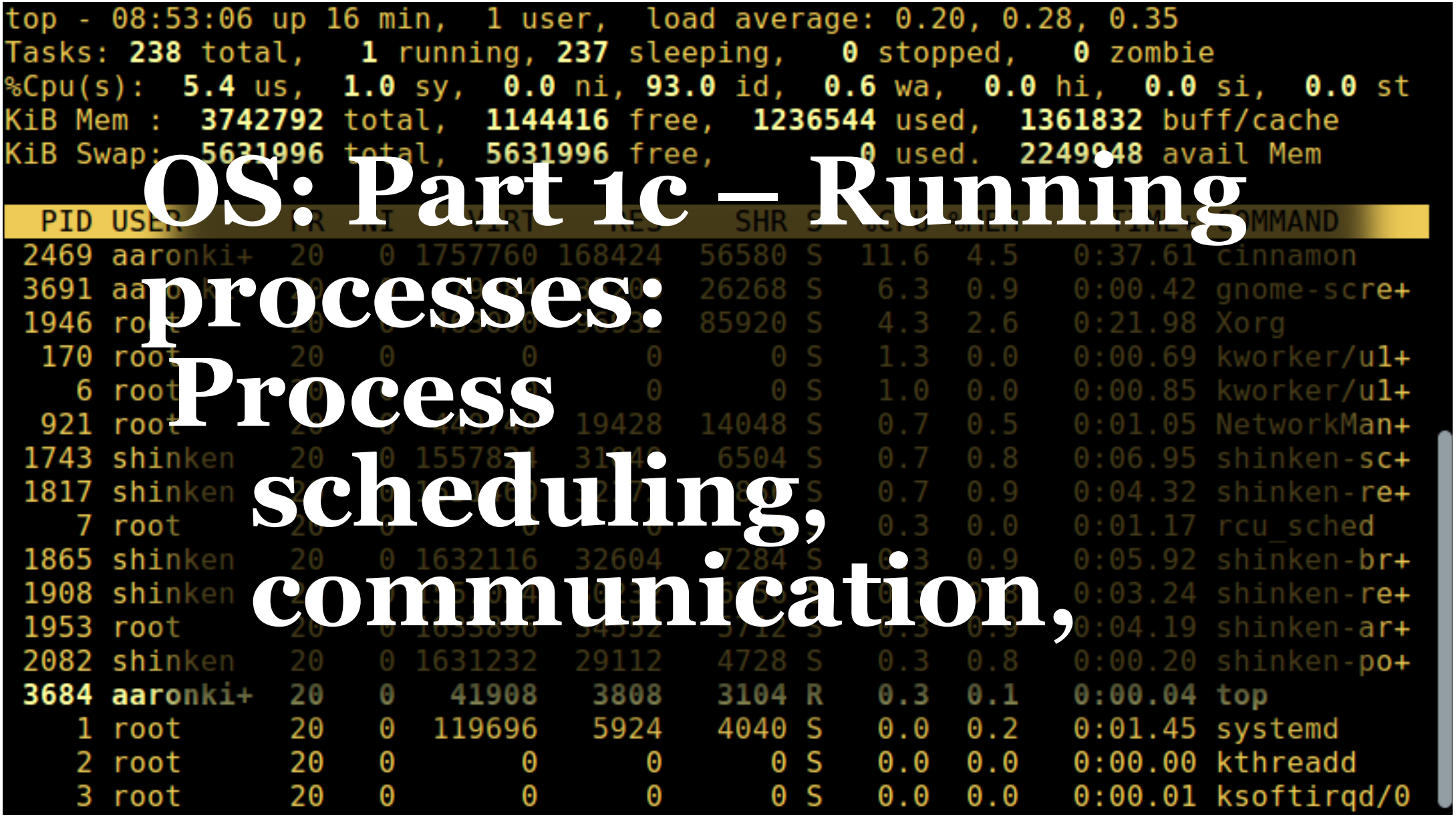
    printf("Hello from
      process, child  PID = %d\n",pid);

    if (pid == -1) {
        perror("fork failed");
        exit(0);
    }
    else if (pid == 0) {
        printf("Forked child, about to be replaced with new process\n");
        execlp("./hello_exec_rep", "hello_exec_rep", NULL);
        exit(1);
    }
    else {

        int status;
        (void)waitpid(pid, &status, 0);
    }
    printf("Bye from the parent process!\n");
    return 1;
}
```

hello_exec_rep.c

```c
int main(void) {
    long x=(long)getpid();
    printf("New replacement process
            pid: %ld\n",x);
    printf("Bye from the new
             process!\n");
    return 1;
}
```



OS: Part 1c – Running processes: Process scheduling, communication,

Process Scheduling



Process State

Only **one process** can be running on any processor at any instant.

A process changes *state* as it executes:

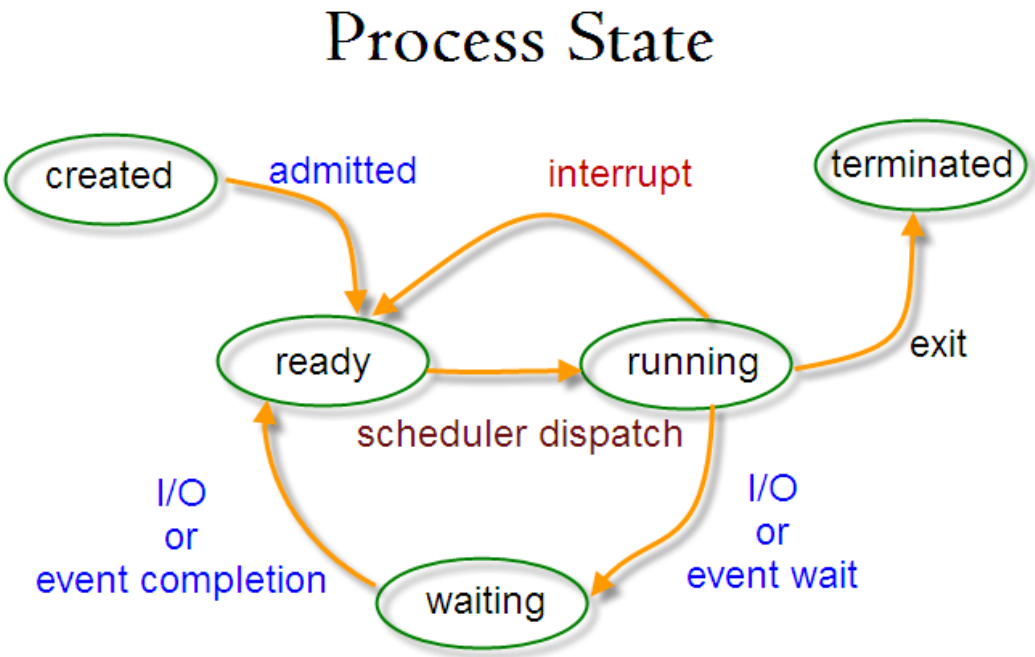
New/created: The process is being created

running: Instructions are being executed

waiting: The process is waiting for some event to occur

ready: The process is waiting to be assigned to a processor

terminated: The process has finished execution



A process state diagram

Context switching

When a process is running, the system is said to be

in the context of that process.

When the kernel decides to execute another process, it performs a

context switch,

causing the system to execute in a different process' context.

For a **context switch,** the kernel has to **save** enough **information** so that it can switch back to the earlier process and continue executing **exactly where it left off.**

When a process executes a **system call** and has to change from *user* to *kernel mode*, the kernel also has to save enough information so that it can later return to user mode and continue executing the program.

Aside: user mode versus kernel mode

In *kernel mode* processes have **unrestricted access** to the **hardware**.

In *user mode* (non-privileged mode for user programs) a process has **limited access** to the CPU and the memory.

- A system call allows a user process to access system resources.

Context switching

When a process is running, the system is said to be **in the context of that process**.

When the kernel decides to execute another process, it performs a **context switch**, causing the system to execute in a different process' context.

For a **context switch,** the kernel has to **save** enough **information** so that it can switch back to the earlier process and continue executing **exactly where it left off**.

When a process executes a **system call** and has to change from *user* to *kernel mode*, the kernel also has to save enough information so that it can later return to user mode and continue executing the program.

How is this done?

CPU context switching

Context switch steps:

- **Save** current process to PCB
- Decide which **process to run next**
- **Load** new process from PCB

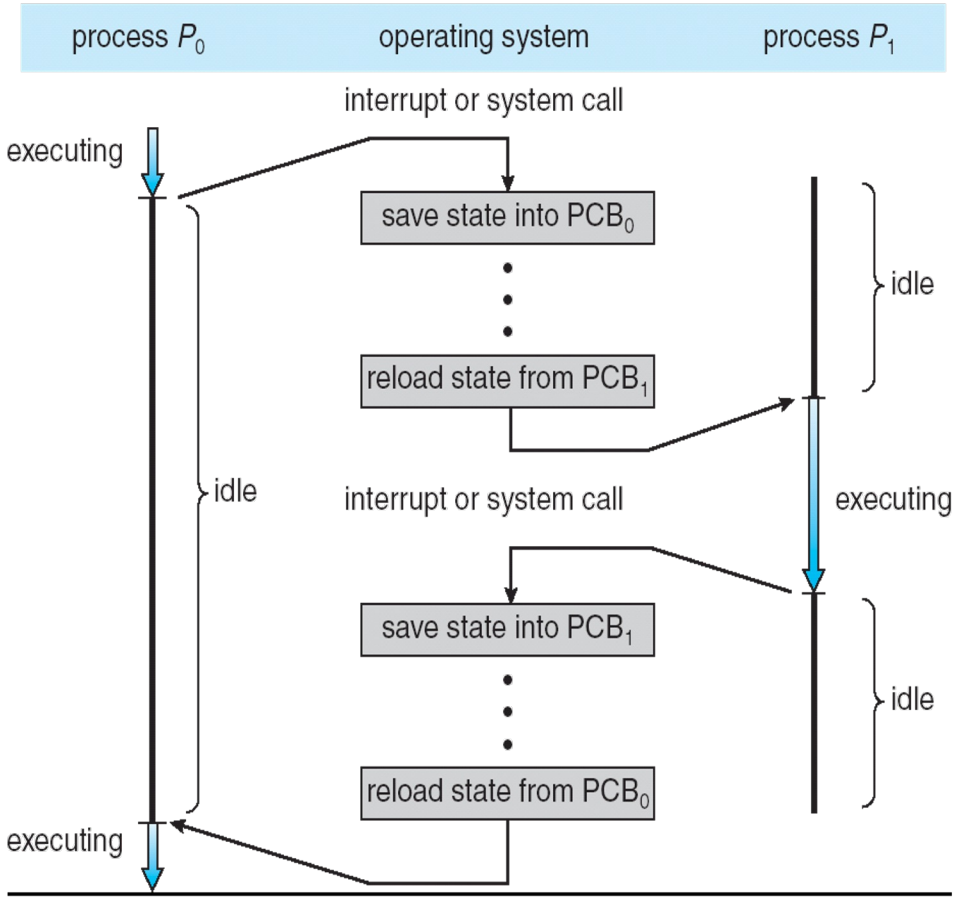
Context switch should be **fast**, because it is overhead: the **system does no useful work** while switching processes.

The more complex the OS and the PCB ➔ the longer the context switch

Time taken is dependent on hardware support:

hardware designers try to support routine context-switch actions like saving/restoring all CPU registers by one pair of machine instructions.

Some hardware provides multiple sets of registers per CPU ➔ multiple contexts loaded at once



Typical context-switching speed is several microseconds

Process Scheduling Queues

Scheduling is used to determine which process next gets access to CPU.

- **Job queue** – set of all processes in the system.
- **Ready queue** – set of all processes residing in main memory, ready and waiting to execute.
- **Device queues** – set of processes waiting for a particular I/O device.

A process scheduler selects among available processes for next execution on CPU.

Processes **migrate** among the various queues.

Queues generally implemented as a doubly-linked list.



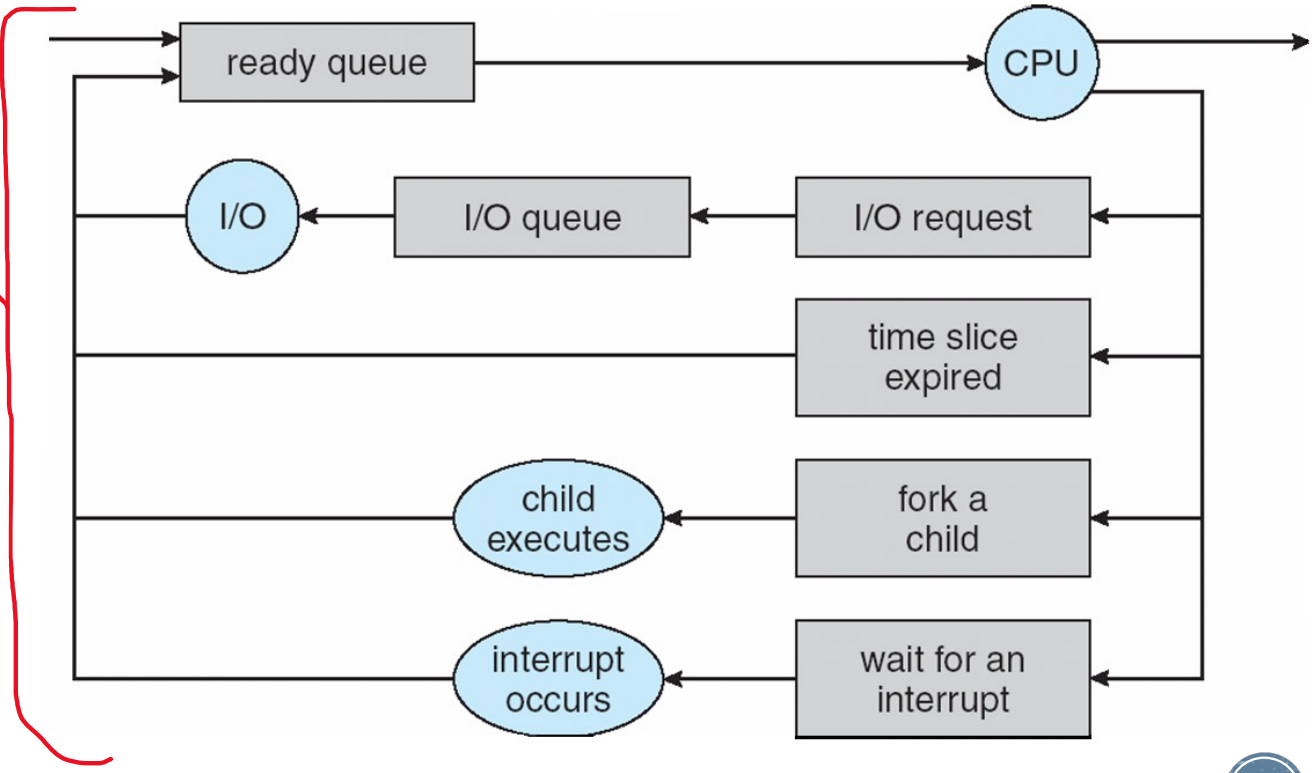
Process scheduling

Queueing diagram represents queues, resources and flow between them.

Circles are **resources**,

arrows indicate **flow**.

New processes start in ready queue until **dispatched.**



Multitasking in mobile operating systems

Mobile phones require operating systems with sophisticated resource management, multi-tasking, security and failure isolation.



iOS

iOS

Strongly **restricts background execution** for user apps

The app in the foreground interacts directly with the user

Background activity is limited and tightly controlled

Certain categories of work may continue in the background, such as audio playback, VoIP, and location

Advantages: better battery life, predictable performance and security.



ANDROID

Android

Foreground and background work is generally more flexible

Background tasks are commonly handled by services

A service can perform work without a user interface

Advantages: greater flexibility, customizable.

Interprocess communication (IPC)

Processes within a system may be independent or **cooperating**.

- **independent** process: does not share data with other processes.
- **cooperating** process communicates with other processes.
  - Information sharing
  - Computation speedup (parallelization)
  - Modularity

Main IPC models

- **Shared memory** — processes communicate by reading/writing a shared region of memory
- **Message passing** — processes communicate by sending data through a channel

IPC Models: Shared memory

OS normally prevents one processes from accessing another process's memory.

With shared memory, the OS creates a region that multiple processes are allowed to access.

- Processes communicate by reading and writing to shared region
- Fast, but requires synchronization

e.g. POSIX shared memory In UNIX, communicating **threads** in Java (CSC2002S).

Message passing: Pipes

- a **unidirectional, transient data channel** between two processes.
- One process writes to one end of the pipe, another reads from the other end.
- Often used to connect the output of one process to the input of another
- Common in shell pipelines, e.g.

```
cat file1.txt | sort
```

Explicit Message passing

Processes communicate by explicitly sending and receiving messages

Basic operations:

```
send(message)
receive(message)
```

Can be synchronous or asynchronous

Useful when processes should remain isolated.

Message passing with Sockets

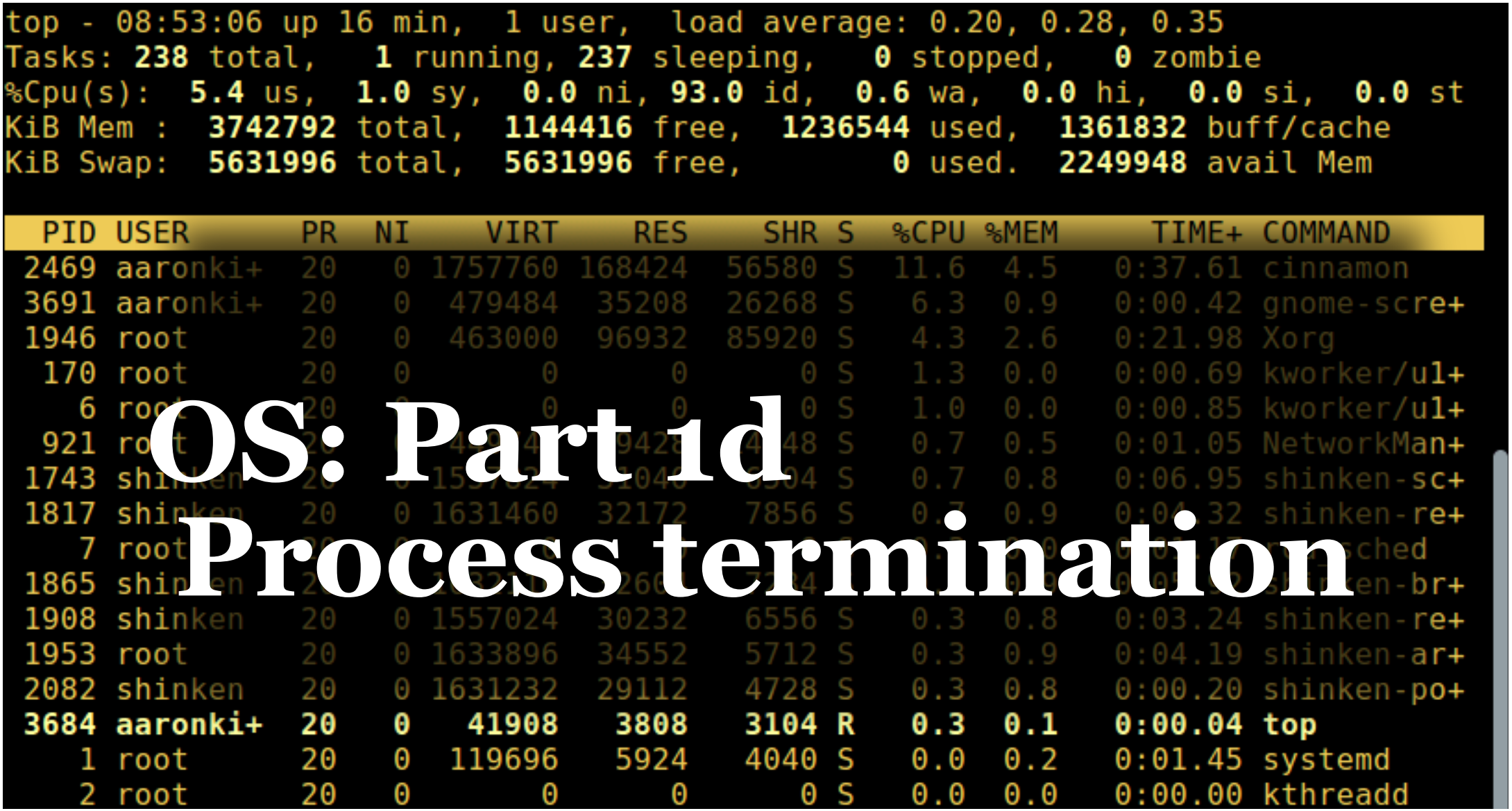
The sockets are endpoints for bidirectional communication

- Can be used for communication on the same machine or across a network
- Common in client-server systems
- Identified by an address plus a port
- web servers listen on well-known ports such as 80 or 443

Message passing: Remote Procedure Calls

Remote Procedure Call (RPC)

- Lets a program invoke a function in another process, possibly on another machine
- Makes remote communication look like a local procedure call
- Built on top of lower-level communication mechanisms such as sockets
- Used in distributed systems and in some OS frameworks, e.g. Android Binder



OS: Part 1d
Process termination

Process Termination



Process termination

A process terminates when it finishes execution or is forced to stop.

Normal exit

Process terminates when it executes its last statement.
`exit()` system call (implicit at end of program or explicit call)
Returns an exit status to its parent

Abnormal termination

Caused by an error, exception, or signal
e.g. invalid memory access, divide-by-zero, killed by another process

On exit, all process resources are **deallocated and reclaimed** by the operating system.
—including physical and virtual memory, open files, and I/O buffer

Process termination

A terminated process is **not fully removed** until its parent, or another adopting process, reaps it. There are two main situations:

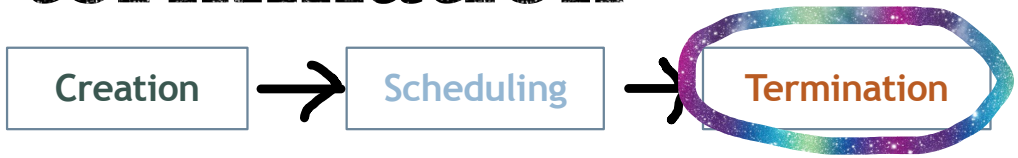
Parent terminates first: Orphan process

- Child process whose parent terminates first
- It is adopted by PID 1 (systemd on Linux)
- The adopting process later reaps it when it terminates
- *Unusual because parents should wait for children to terminate with* **`wait()` / `waitpid()`**

Child terminates first: Zombie process

- A child process that has terminated, but its parent has *not yet collected its exit status*.
- PCB entry remains so the parent can read the termination information
- Removed when the parent calls
  - `wait()` any child or
  - `waitpid()` specific child

Abnormal Process termination



A process can terminate another process via a system call

e.g `TerminateProcess()` in Windows, `abort()` in Unix/Linux

Usually, a terminate system call can be invoked only by a parent process.

Otherwise, a user—or a misbehaving application—could arbitrarily kill another user's processes.

Because a parent needs to know the identities of its children, the identity of the newly created process is passed to the parent on creation.

A parent may terminate the children processes because:

- Child has exceeded allocated resources
- Task assigned to child is no longer required
- The parent is exiting and the operating system does not allow a child to continue if its parent terminates



Creating orphans (and zombies)

To create an orphan process, kill the parent process while the child process is still around.

(*Note this is for illustration purposes - not a good thing to do…)*

hello_orphan.c

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main(void)
{
    long x=(long)getpid();
    printf("This is the start of the
greetings\n");

    printf("Hello, %ld!\n",x);
    fork();
    x=(long)getpid();
    //x+=1;
    printf("Hello, %ld!\n",x);
    fork();
    //x+=1;
    x=(long)getpid();
    printf("Hello! %ld!\n",x);
    sleep(30);
    printf("Goodbye, %ld!\n",x);
    return 0;
}
```



Creating orphans (and zombies)

If child process runs forever, not necessarily killed by OS…

Run, kill parent,

```
ps –f
kill -9 <PID>
```

hello_orphan_perpetual.c

```c
#include <unistd.h>
#include <sys/types.h>
#include <errno.h>
#include <stdio.h>
#include <sys/wait.h>
#include <stdlib.h>
int main(void)
{
    pid_t pid = fork();

    printf("Hello from process, created child with PID = %d\n",pid);

    if (pid == -1) {
        perror("fork failed");
        exit(0);
    }
    else if (pid == 0) {
        while (1) {
            printf("Hi from the child process! Fork returned %d\n",pid);
            sleep(15);
        }
        exit(1);
    }
    else {

        int status;
        printf("Parent sleeping for 30s - check for zombie!\n");
        sleep(30);
        (void)waitpid(pid, &status, 0);
    }
    printf("Bye from the parent process!\n");
    return 1;
}
```

Notes on zombies



The parent process does not kill off zombies - zombies **are already dead.**

What happens is that the zombie child's process ID is finally removed from the process list after the parent calls "wait" (or exits itself).

Zombies exist purely so that the parent can see that the child process has ended.

- Parents check/wait for the child process termination with `(void)waitpid(pid, &status, 0);`
- This wait uses the process ID of the child. If the child finished long before the parent called wait, it's PID could be reallocated and this would be a mess (the parent could be waiting on a non-process or another process).
  - So the Operating System keeps terminated child processes around in the process list as "**zombies**" - purely so that the ID is not reallocated and so a parent can call wait() without problems.